Operating Systems Introduction
MCA

- **What is an Operating System (OS)?**

An Operating System is a program that acts as an interface between the user and computer hardware.

Main goals of OS:

Execute user programs

Make the system easy to use

Use hardware efficiently (CPU, memory, I/O)

- **Computer System Components**

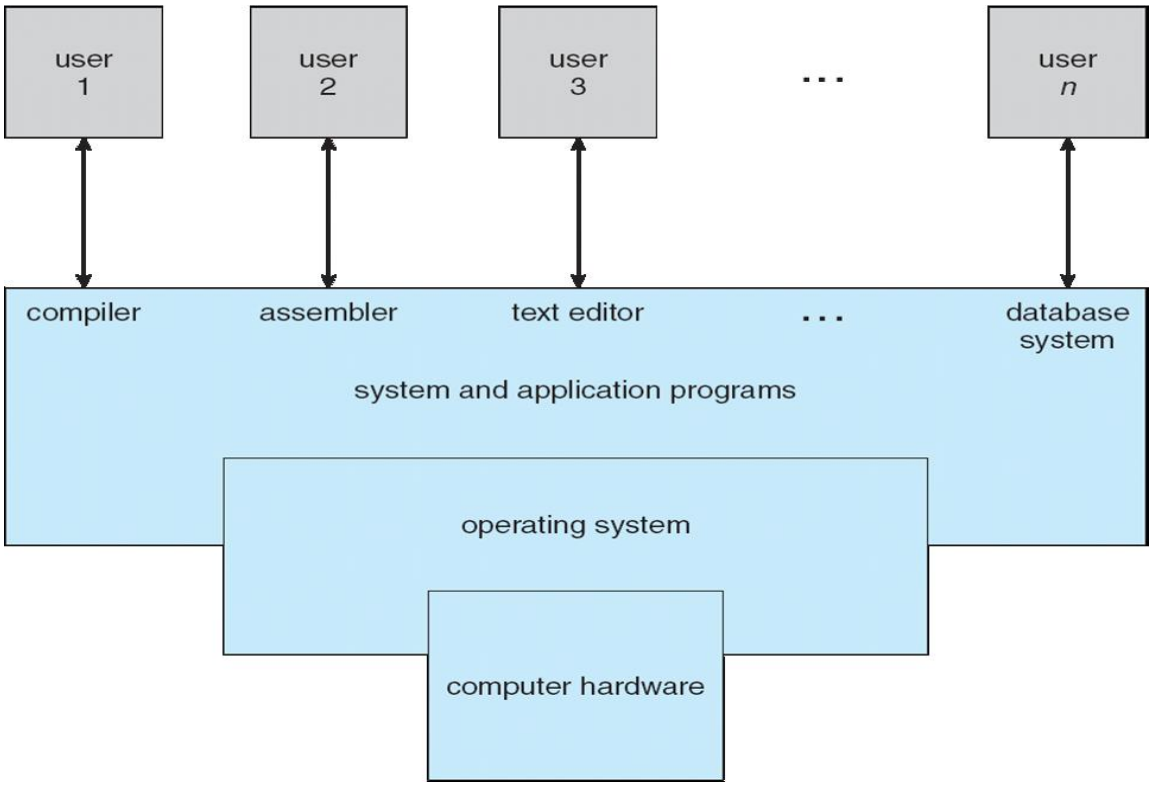
A computer system has four main components:

Hardware – CPU, memory, I/O devices

Operating System – manages and controls hardware

Application Programs – word processor, browser, compiler, DBMS, games

Users – humans, machines, other computers



Operating Systems Introduction
MCA

- **Operating System Definition**

OS acts as a resource allocator

Manages CPU, memory, storage, I/O

Resolves conflicts between users/processes

OS is also a control program

Prevents misuse of hardware

Controls execution of programs

Kernel: Core part of OS that runs all the time

- **Computer Startup (Booting)**

Bootstrap program loads when system starts

Stored in ROM / firmware

Initializes system hardware

Loads OS kernel into memory

- **Computer System Organization**

CPU and device controllers connected via system bus

Devices and CPU execute concurrently

Devices have local buffers

Interrupts notify CPU after I/O completion

- **Interrupts**

Interrupt transfers control to Interrupt Service Routine (ISR)

Interrupt Vector stores addresses of ISRs

Types:

Hardware interrupt (from devices)

Software interrupt (trap/exception)

OS is interrupt-driven

- **I/O Structure**

Two approaches:

CPU waits until I/O completes (inefficient)

CPU continues execution after starting I/O (efficient)

System call allows user to request OS services

Device-status table tracks all I/O devices

- Storage Basics

Bit: smallest unit (0 or 1)

Byte: 8 bits

Word: CPU's natural data size

Storage units:

1 KB = 1024 bytes

1 MB = 1024^2 bytes

1 GB = 1024^3 bytes

1 TB = 1024^4 bytes

- **Storage Structure**

Main Memory (RAM):

Directly accessed by CPU

Volatile

Secondary Storage:

Hard disks, SSDs

Non-volatile

SSDs are faster and increasingly popular

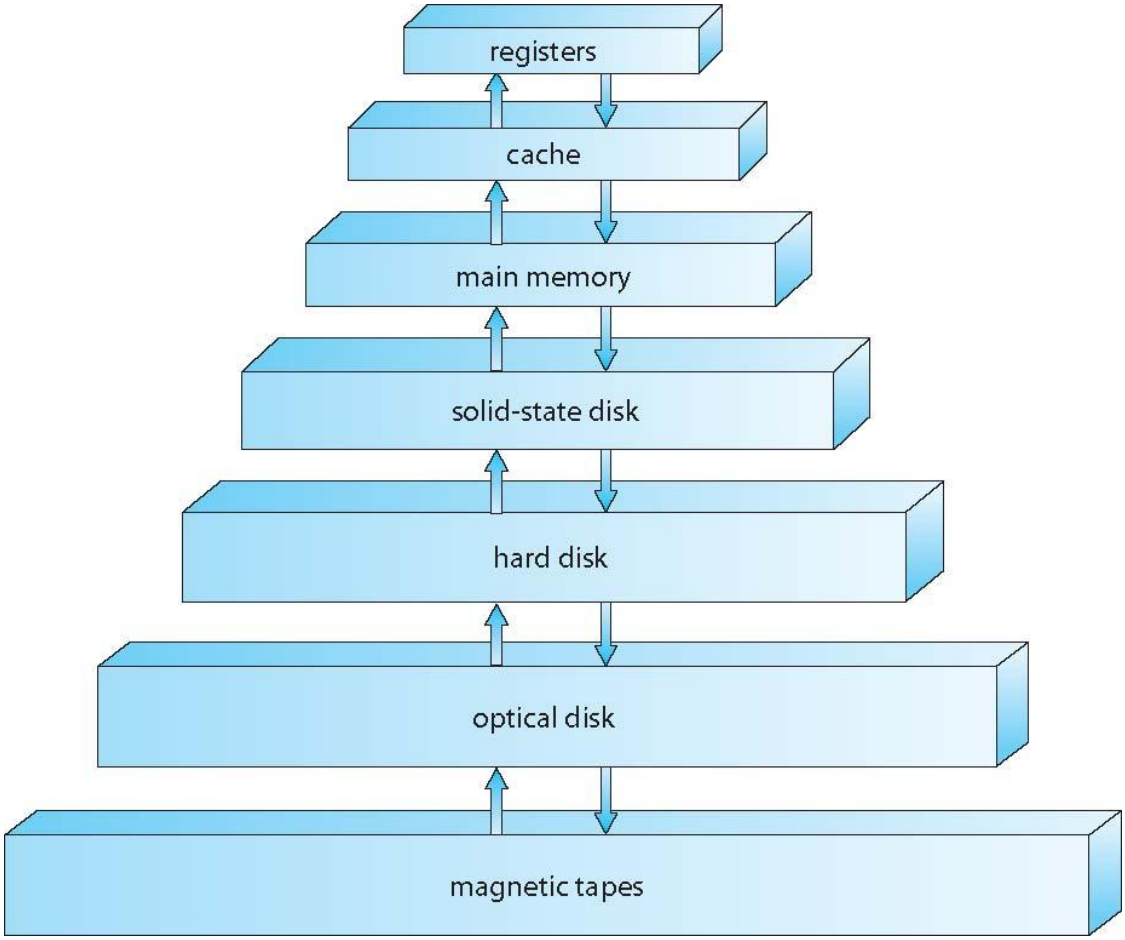
- Storage Hierarchy

Organized by

Speed

Cost

Volatility



Order (fast → slow):

Registers → Cache → Main Memory → SSD → HDD → Optical → Tape

Caching improves performance by storing frequently used data

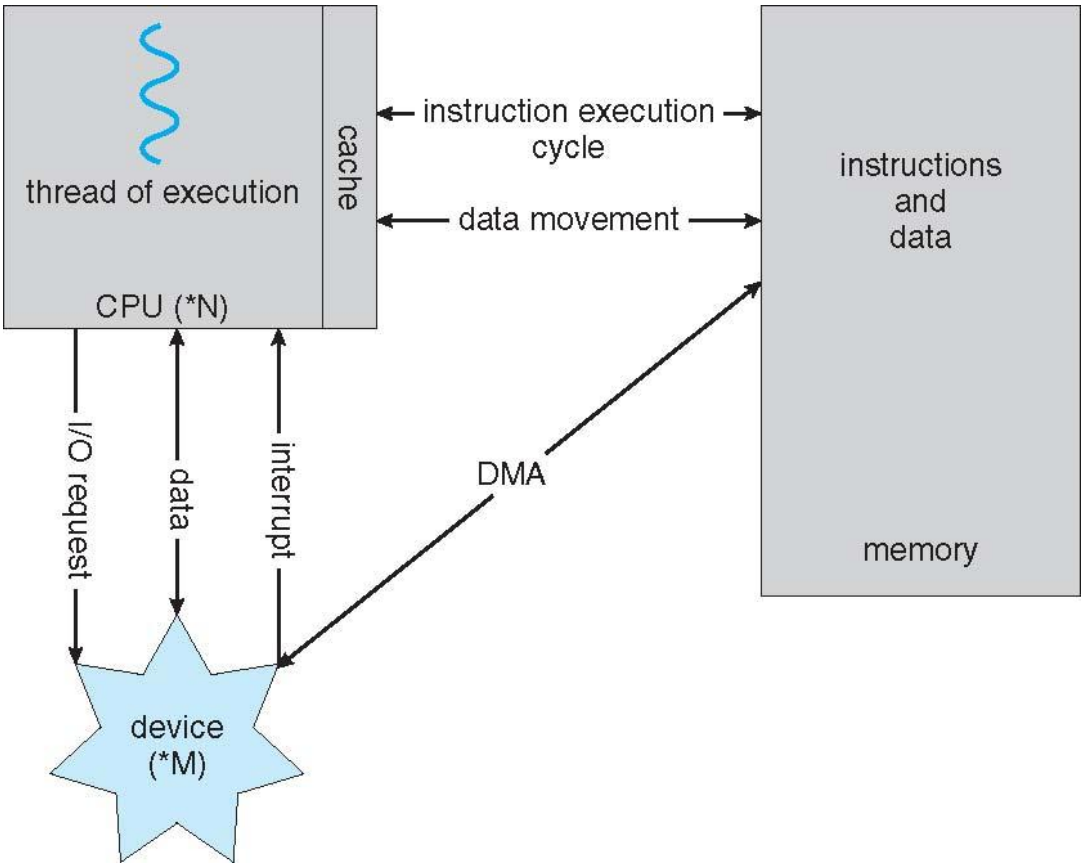
Direct Memory Access (DMA)

Used for high-speed I/O devices

Data transferred directly between device and memory

Reduces CPU involvement

One interrupt per block instead of per byte



- **Computer System Architecture**

Single-processor systems

Multiprocessor systems (parallel systems)

Advantages:

Higher throughput

Better reliability

Cost efficiency

Types:

Asymmetric Multiprocessing (AMP)

Symmetric Multiprocessing (SMP)

- **Multicore and Clustered Systems**

Multicore: multiple cores on one CPU chip

Clustered systems: Multiple computers working together

Share storage (SAN)

Used for high availability and HPC

- **Multiprogramming & Time Sharing**

Multiprogramming:

Keeps CPU busy by switching between jobs

Time sharing (Multitasking)

CPU switches rapidly among processes

Interactive computing

Response time < 1 second

Uses CPU scheduling, swapping, virtual memory

- **Operating System Operations**

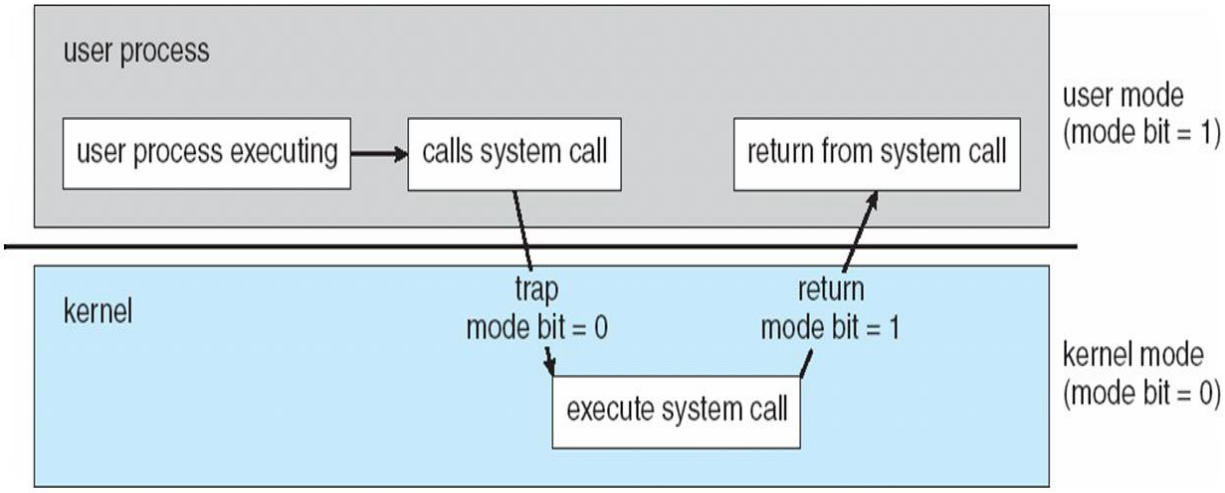
OS works using interrupts

Dual-mode operation:

User mode

Kernel mode

Operating Systems Introduction
MCA



Privileged instructions run only in kernel mode

System calls switch from user → kernel mode

- Timer

Prevents infinite loops

Generates interrupt after fixed time

Helps OS regain control of CPU

- **Process Management**

Process: program in execution

Needs resources: CPU, memory, I/O, files

Types:

Single-threaded process

Multi-threaded process

OS responsibilities:

Create & delete processes

Synchronization

Communication

Deadlock handling

- Memory Management

Determines what is stored in memory and when

Improves CPU utilization

Tasks:

Track memory usage

Allocate & deallocate memory

Swap processes in/out of memory

- Storage & File Management

OS provides logical view of storage using files

File system responsibilities:

Create/delete files & directories

Access control

File backup

Mapping files to disk

- Mass Storage Management

Disk performance affects system speed

OS handles:

Free-space management

Disk scheduling

Storage allocation

Tertiary storage: optical disks, magnetic tapes

Operating Systems Introduction
MCA

- **I/O Subsystem**

Hides hardware complexity

Manages:

Buffering

Caching

Spooling

Uses device drivers

- **Protection and Security**

Protection: controls access to resources

Security: protects system from attacks

Uses:

User IDs

Group IDs

Access control

Privilege escalation

- **Kernel Data Structures**

Used to manage OS resources

Common structures:

Linked lists (singly, doubly, circular)

Trees (binary search tree)

Hash maps

Bitmaps

Linux defines these in kernel header files

- **Operating System Services**

OS provides an environment to execute programs

Offers services to users and programs

Services are broadly divided into:

User oriented services

System-oriented (resource management) services

- **User-Oriented Operating System Services**

User Interface (UI)

Allows user to interact with the system

Types:

CLI (Command Line Interface) – text commands

GUI (Graphical User Interface) – windows, icons, mouse

Touchscreen Interface – gestures, virtual keyboard

Batch Interface – no direct user interaction

Program Execution

OS loads program into memory

Executes the program

Handles:

Normal termination

Abnormal termination (errors)

Operating Systems Introduction
MCA

I/O Operations

Programs may need input/output

OS provides controlled access to:

Files

I/O devices (keyboard, printer, disk)

File-System Manipulation

OS allows programs to:

Create and delete files

Read and write files

Create and remove directories

Search and list files

Manage file permissions

Communications

Processes exchange information:

On the same system

Across networked systems

Methods:

- Shared memory
- Message passing

Operating Systems Introduction
MCA

Error Detection

OS continuously monitors errors in:

CPU and memory

I/O devices

User programs

OS takes corrective action

Debugging tools help programmers

System-Oriented OS Services

Resource Allocation

Allocates resources when multiple jobs/users run together

Resources include:

CPU

Memory

Storage

I/O devices

Accounting (Logging)

Keeps track of:

Resource usage

Which user used how much and what

Operating Systems Introduction
MCA

Protection and Security

Protection: Controls access to resources

Security: Prevents unauthorized access

Includes:

User authentication

Protection of files, memory, devices

- Command-Line Interpreter (CLI)

Allows users to enter commands directly

Can be:

Part of kernel

Separate system program (shell)

Multiple shells possible

Commands may be:

Built-in

External programs

- Graphical User Interface (GUI)

Uses windows, icons, menus, mouse

Easier for beginners

Examples:

Windows

macOS

Linux GUIs (GNOME, KDE)

- Touchscreen Interfaces

Used in mobile and tablet devices

Features:

Gesture-based input

Virtual keyboard

Voice commands

System Calls

System calls are programming interface to OS services

Used by programs to request OS operations

Written mostly in C/C++

Usually accessed via APIs, not directly

Common APIs:

Win32 API – Windows

POSIX API – UNIX, Linux, macOS

Java API – JVM

- **System Call Implementation**

Each system call has a unique number

System-call interface:

Identifies the call

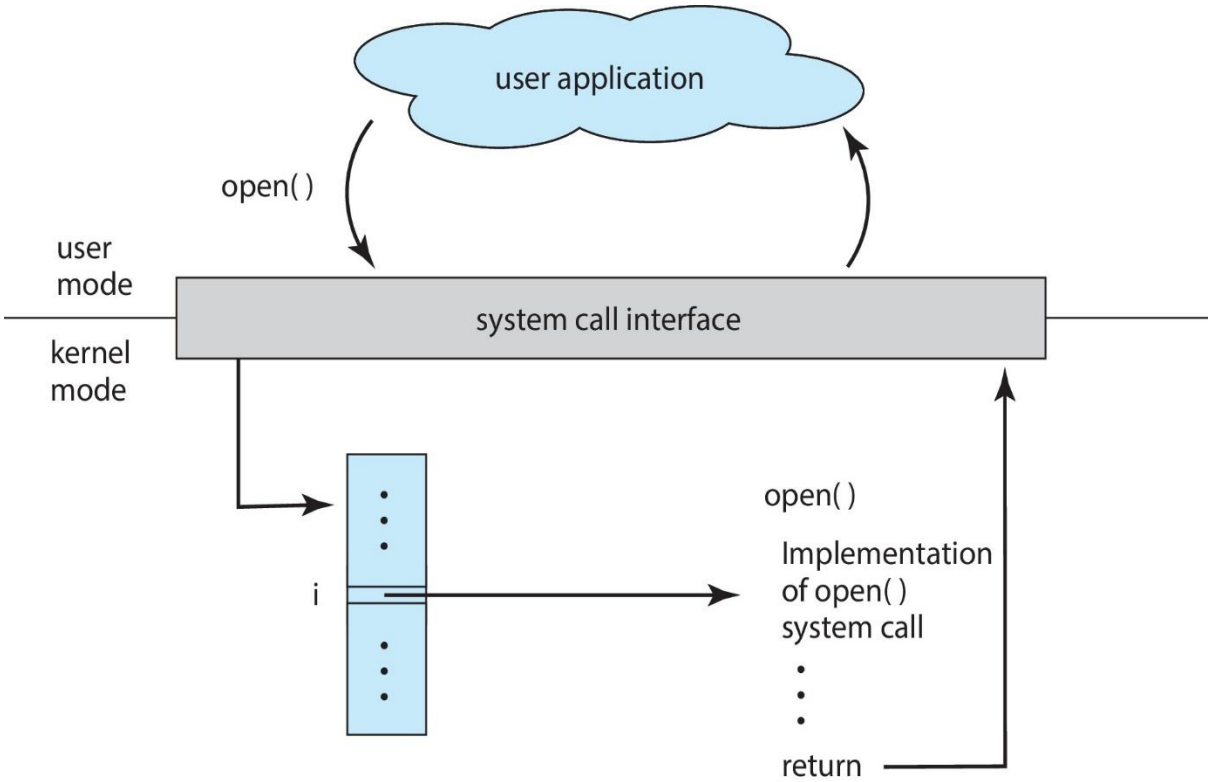
Invokes kernel service

Returns result

API hides internal OS details from programmers

Operating Systems Introduction
MCA

- System Call Parameter Passing



Three methods:

Registers

Memory block / table (used in Linux)

Stack

- **Types of System Calls**

Process Control

Create, terminate process

Load, execute

Wait, signal

Allocate/free memory

Debugging support

- **File Management**

Create, delete files

Open, close

Read, write

Set/get file attributes

- **Device Management**

Request/release device

Read/write device

Set/get device attributes

Information Maintenance

Get/set time and date

Get/set system and process information

- **Communication**

Create/delete connections

Send/receive messages

Shared memory operations

- **Protection**

Control access to resources

Set permissions

Allow or deny access

- **System Programs (System Services)**

Provide a convenient environment for users

Categories:

File manipulation

Status information

Programming support

Program loading & execution

Communication

Background services

- **Background Services (Daemons)**

Start at system boot

Run in background

Examples:

Printing

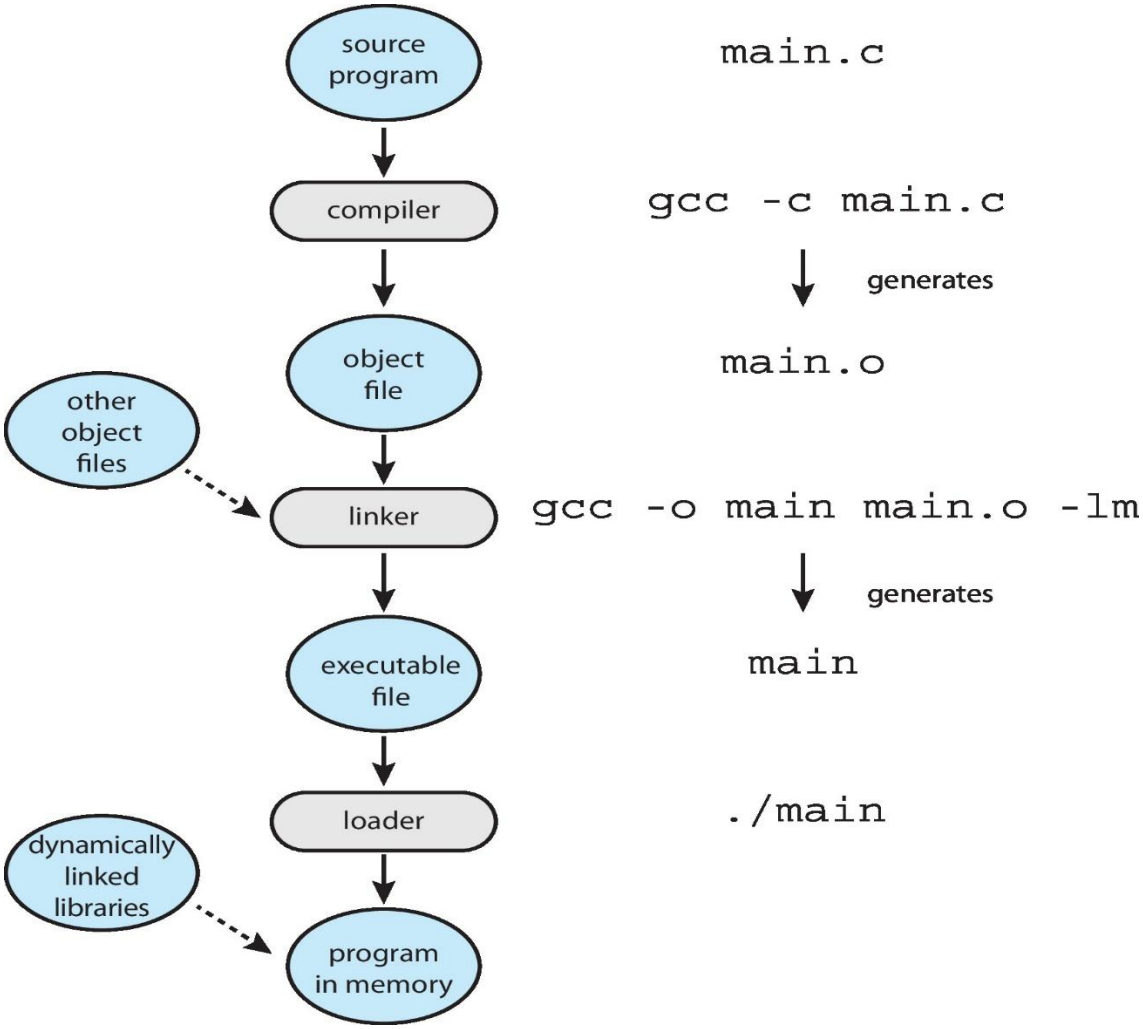
Logging

Scheduling

Run in user mode, not kernel mode

Operating Systems Introduction
MCA

- **Linkers and Loaders**



Linker

Combines object files into one executable

Adds required libraries

Loader

Loads executable into memory

Performs relocation

Starts program execution

Dynamic Linking

Libraries loaded at runtime

Shared among processes

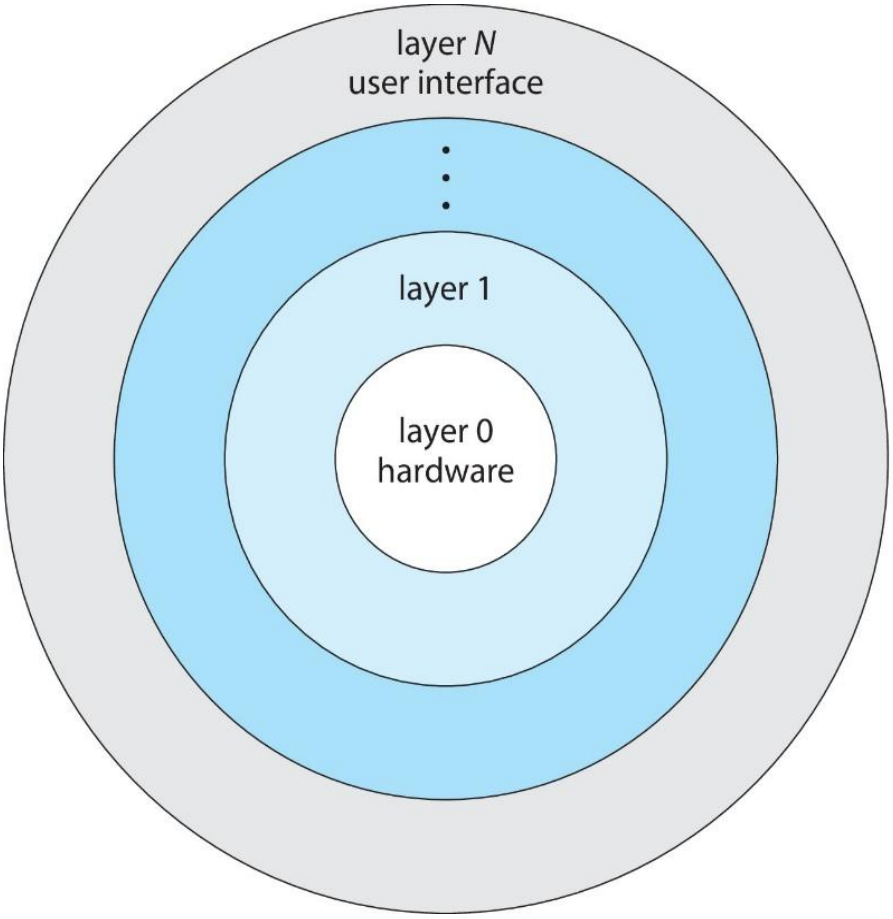
Saves memory

- Operating System Structure

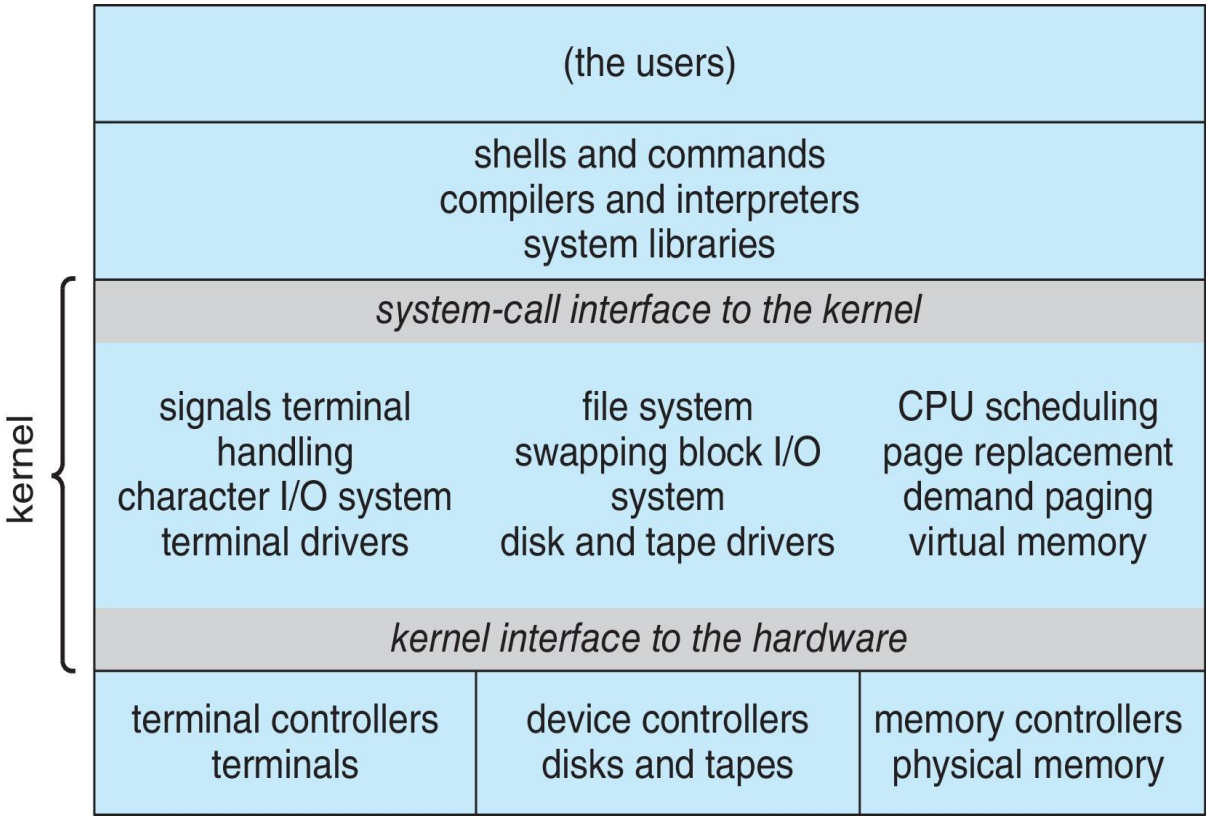
Different OS design approaches:

Simple Structure – MS-DOS

Layered Structure



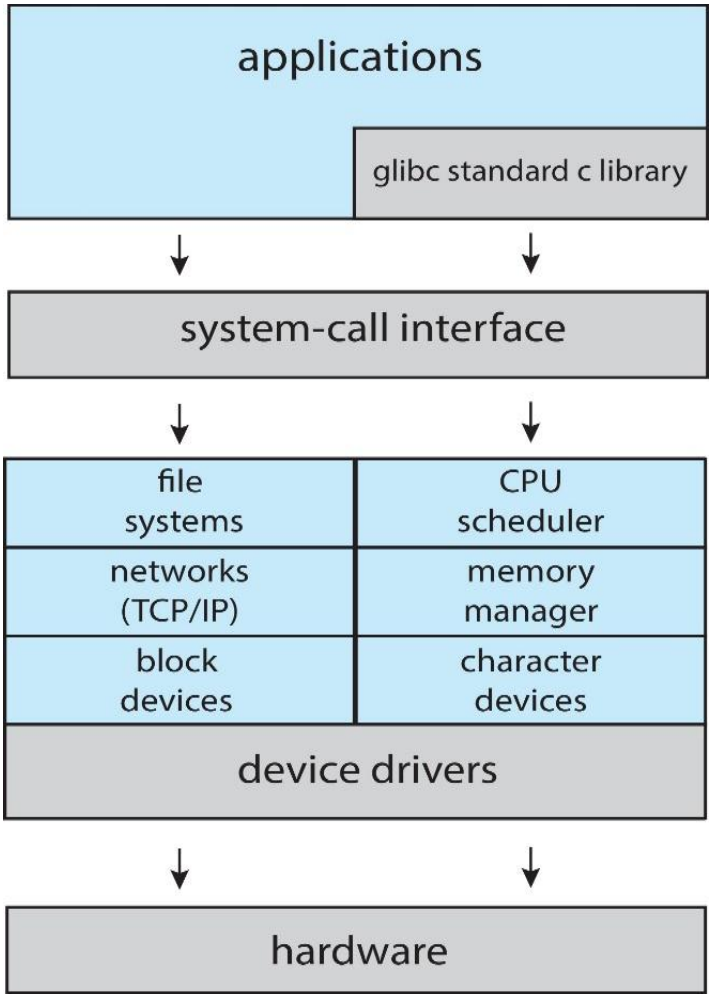
Monolithic Structure – UNIX



Operating Systems Introduction
MCA

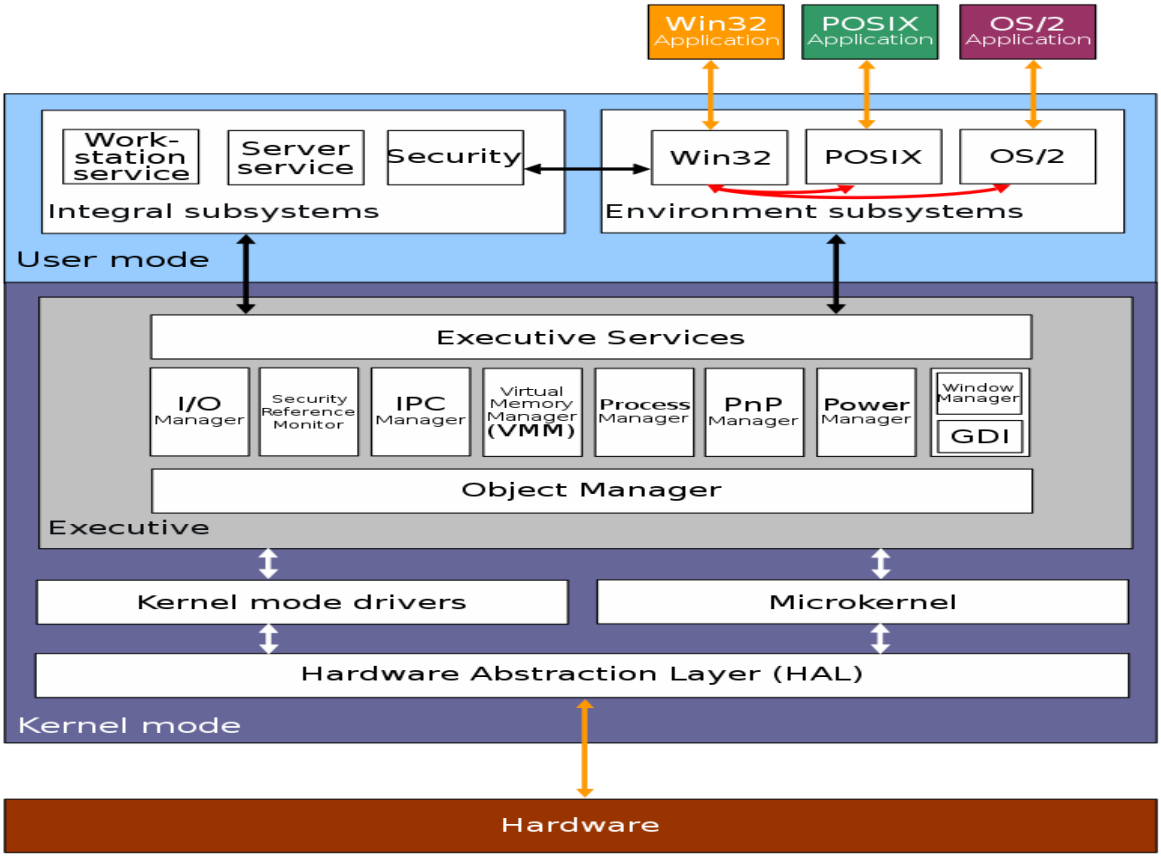
Linux System Structure

Monolithic plus modular design



Operating Systems Introduction
MCA

Windows System Structure



- **Operating System Debugging**

Debugging = finding and fixing bugs

OS generates:

Log files

Core dumps (application crash)

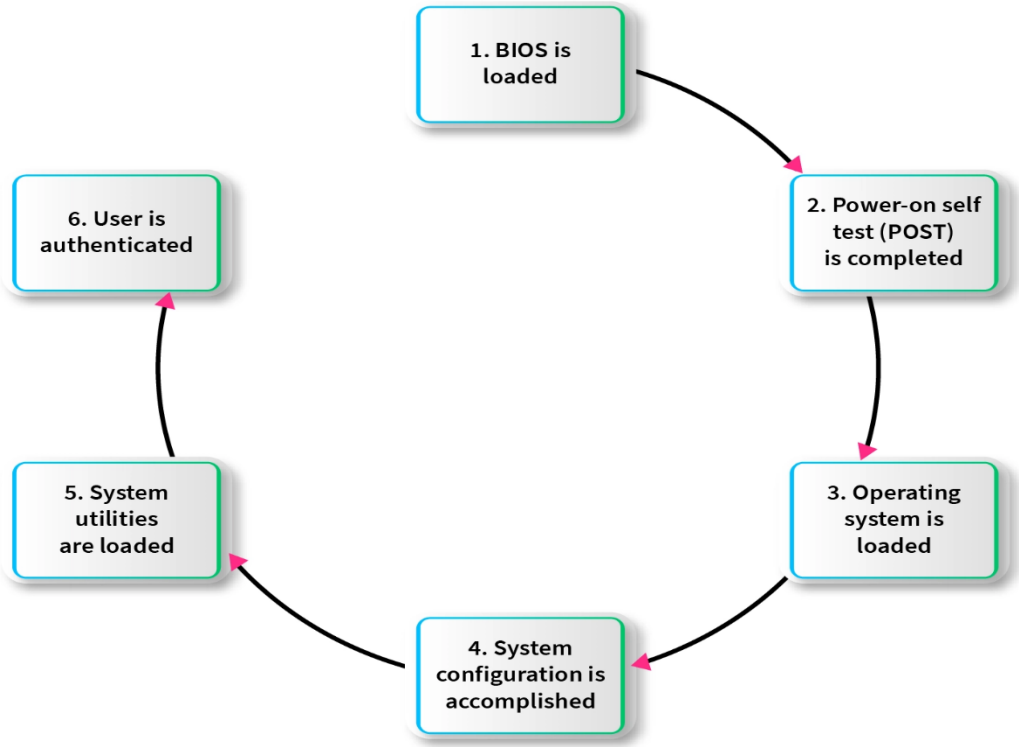
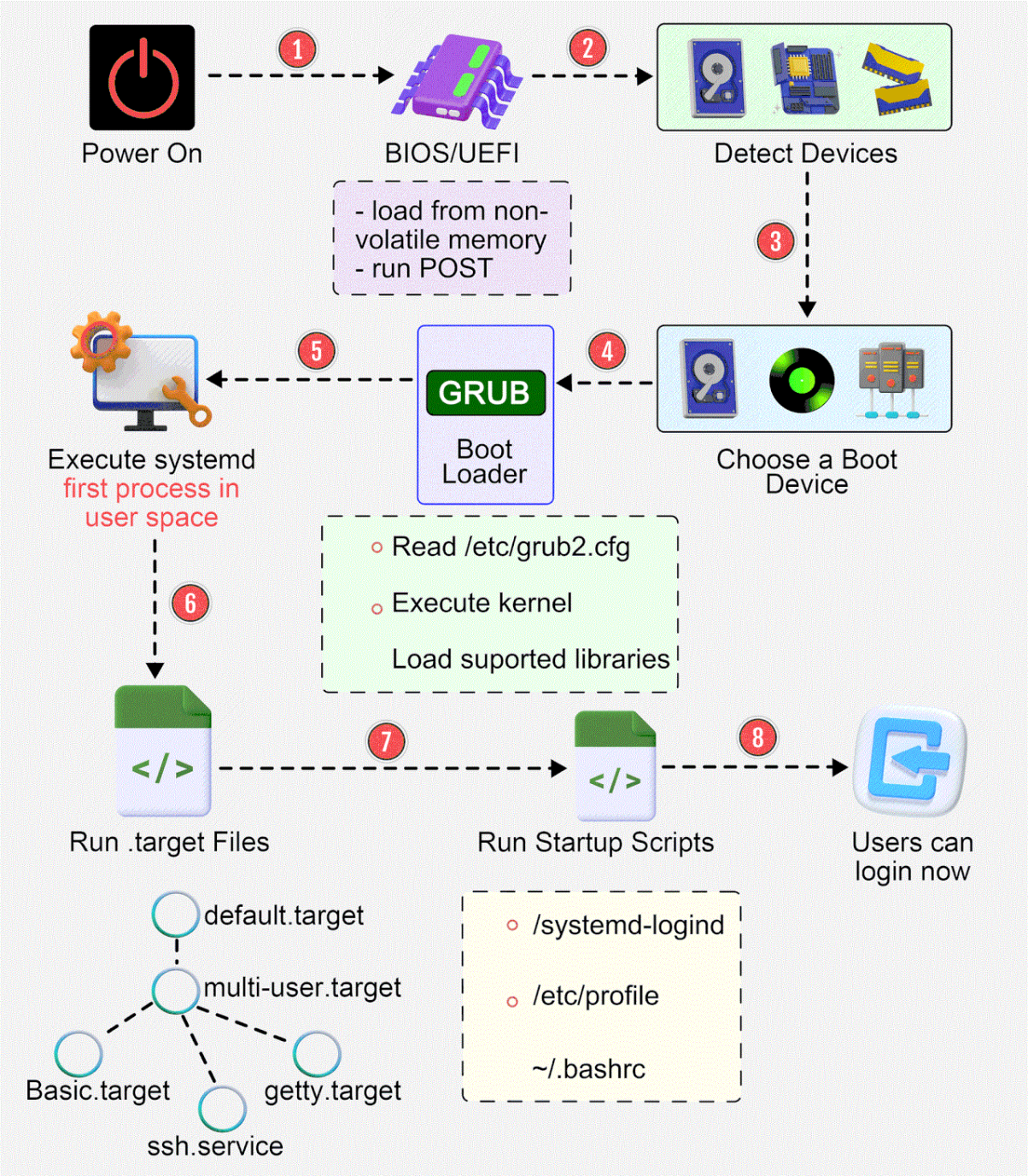
Crash dumps (OS failure)

Performance tuning:

Tracing

Profiling

Booting


1. BIOS is loaded
2. Power-on self test (POST) is completed
3. Operating system is loaded
4. System configuration is accomplished
5. System utilities are loaded
6. User is authenticated

Operating Systems Introduction
MCA